When Your LLM Is Wrong, What Kind of Wrong Is It?

A diagnostic ladder — structure, meaning, context, grounding, environment — for the conversation between engineers and the people who use what they build.

Lodewijk Bonebakker*

April 2026

A junior developer on your team has just shipped a small change. They asked the language model to write a function that filters customer records by region and returns the names sorted by purchase total. The function passed code review. It runs without error. The output is wrong, and nobody can quite explain why.

This is, at this point, a nearly universal experience. The model produces something that looks right. It is wrong in ways that are hard to articulate. The team falls back on the practitioner vocabulary that has accumulated over the last five years: someone says *hallucination*, someone else says *prompt brittleness*, a third person mentions *lost in the middle*. Each of these terms picks out a real phenomenon. None of them, considered as a set, forms a coherent diagnostic procedure.

The vocabulary the field has built for itself is sharp at the level of mechanism (cross-entropy training optimizes plausibility, not truth) and dull at the level of human experience (the answer just felt off). The gap between these two registers is where most production incidents are diagnosed badly, communicated unclearly, or fixed in the wrong place. This article proposes a closing of that gap: a five-axis model of LLM error, ordered from the most mechanical (what the model is best at) to the most human (what only the user can supply), that practitioners can hold in their heads and walk through in less than a minute.

The five axes are *structure*, *meaning*, *context*, *grounding*, and *environment*. They form a ladder. At the bottom, the model is at its strongest and failures are rare. At the top, the model is, in a precise sense, at the mercy of the human, and failures are inevitable unless the human compensates. The model is not a theory. It is a vocabulary, with a procedure attached, designed to be useful in conversations with stakeholders who do not have a year to spend on neural-network internals.

Why a layered model, and why now

The instinct to organize language around layers is older than the language model. Linguists since the early twentieth century have distinguished syntax (the rules of well-formedness), semantics (what an utterance means), and pragmatics (the situational effect it has). Each layer has its own rules and its own diagnostic vocabulary. When we ask whether a sentence makes sense, we are asking different questions at each level, and we know which questions belong on which level.

Two strands of recent work make this organization newly relevant. Stevan Harnad's 1990 paper *The Symbol Grounding Problem* [1] asked how the symbols inside a computer could come to refer to anything outside the computer at all. Emily Bender and Alexander Koller's 2020 paper *Climbing towards NLU* [2] sharpened the question: a system trained only on

*Drafting assistance: Claude Opus 4.7 (Anthropic). The author chose the framework, scope, and voice; the model produced prose under direction; the author edited. Errors that survived this process are the author's.

linguistic form cannot, by that training alone, learn meaning, because meaning is a relation between form and the world that the form alone does not contain. The grounding problem is not a quirk of LLMs; it is the structural fact that the cross-entropy objective they are trained on cares about the plausibility of the next token, not its truth. The practitioner literature on hallucination [3] is, in the end, a practitioner-readable restatement of the same problem.

The five-axis model proposed here is an attempt to operationalize the grounding distinction in a way practitioners can use. It places structure and meaning on the model side, environment on the human side, and context and grounding in between. The boundary between what the model can do and what the human must supply turns out to be the most useful division to teach.

Structure: is the output well-formed?

A model's output is structurally correct if it satisfies the formal constraints of the format being requested. A JSON object that parses; a Python script that imports; a SQL query the database accepts; a LaTeX document that compiles. Structure is the most mechanical of the five axes and is, for that reason, where modern LLMs perform best.

A structural failure is the kind a parser can catch. The fix lives in the decoding stack — JSON-mode, schema-validated sampling, regex-constrained generation, finite-state-machine guards over the token distribution. Frontier models in 2026 are good enough at structure that engineering teams who still have to debug structural failures regularly are usually doing something unusual: deeply nested formats, formats outside the training distribution, or outputs long enough to drift mid-stream.

The diagnostic risk on this axis is the inverse of the failure rate. Structural correctness is so easy to verify that practitioners over-rely on it. A syntactically valid JSON object can still mean the wrong thing, say the wrong thing, and refer to a thing that does not exist. A practitioner who confirms structure and stops has confirmed the least interesting property of the output.

Meaning: is the output internally coherent?

A structurally correct output can fail at the level of internal coherence. The model produces a paragraph whose first sentence contradicts its third. The function it writes declares a return type of `list[str]` but actually returns tuples. The argument relies on a premise the same output has elsewhere denied. These failures are detectable by careful reading without any appeal to the world; they are failures of the output considered as a self-contained artifact.

The mechanism is the same one that produces fluent text in the first place. A language model trained to predict the next token given the prefix optimizes for the plausibility of each token in isolation. Plausibility is local. A token can be plausible at every position in a sequence and still produce, at the end, a global contradiction. The model has no internal mechanism that checks the output as a whole against itself. Coherence is a side-effect of training, not an enforced invariant.

This is the axis that benchmarks tend to overstate. A model that scores 85 on a multiple-choice reading-comprehension test may produce, in free generation, paragraphs that contradict themselves in subtle ways. Domain expertise unmasks this: the technical reader who knows the territory catches inconsistencies the casual reader does not. Remedies are inference-time — ask the model to check its own work, sample multiple completions and compare them,

use chain-of-thought to externalize the reasoning. These all trade inference compute against meaning reliability.

Context: is the output appropriate for the situation?

A model can produce a structurally valid, internally coherent paragraph that nevertheless answers the wrong question. A user asks for a summary of a research paper for a non-specialist audience and receives a paragraph dense with jargon. A user asks a quick yes-or-no question and receives a five-paragraph essay. A user asks a clarifying question and receives a generic safety disclaimer. None of these are factually wrong. All of them are wrong for the situation.

The mechanism is the gap between what the user knows and what the prompt encodes. The model has access only to the tokens in its context window. Anything the user knows about themselves, their audience, or their goals that is not written into the prompt does not exist for the model. Few-shot prompting attempts to encode situational expectations through examples. System prompts attempt to fix the situation up front. Alignment training shapes defaults — politeness, refusal, length, formality — but defaults are defaults, not the situation.

Context failures are easy to recognize and harder to fix. The user can usually point at the answer and say what is wrong with it ("too long," "too technical," "misses the actual question"). The fix is almost always to give the model more of the situation: who you are, who you are writing for, why you need the answer, what would count as a good one. The asymmetry between the ease of recognition and the difficulty of fix is the diagnostic signature of a context failure.

Grounding: does the output connect to reality?

The fourth axis is whether the output connects to verifiable reality or has drifted into plausible-sounding fabrication. The practitioner literature calls this hallucination: the model produces a fluent, confident, well-formed claim that does not correspond to any fact in the world.

Grounding failures are uniquely dangerous because no internal reading of the output reveals them. A hallucinated citation has the same structural form as a real one. A hallucinated statistic has the same surface plausibility as a true one. The check must come from outside the text. The model has produced the most probable continuation of its prompt; the most probable continuation is sometimes true, sometimes false, and the model cannot, on its own, tell the two apart.

This is the place where the symbol-grounding literature is most directly useful. Harnad's question — how can symbols mean anything? — has, for current architectures, an honest answer: they cannot, on their own. The remedy at the system level is retrieval. Replace the question "what is the most probable thing to say" with "what is the most probable thing to say given these specific sources." Grounding is no longer the model's problem; it is the retriever's problem and the prompt's problem. Tool use extends the same idea to verification: a model that can call a calculator delegates arithmetic; a model that can query a database delegates fact lookup. The pattern is to take the work that demands grounding and move it outside the language model entirely.

The diagnostic discipline is to treat any factual claim by the model as a hypothesis. If the claim matters, verify it. If it cannot be verified, do not use it. The careful practitioner

builds workflows that make verification cheap (retrieval, citations, linked sources) so that grounding failures are catchable by construction.

Environment: did the prompt encode the situation?

The fifth axis sits at the human end of the ladder, and is the hardest to see because, by construction, it is invisible. An environment failure occurs when the answer is wrong because the prompt did not contain enough of the situation for the question to be answerable. The user knows things — about themselves, their domain, their constraints, the local conventions of their organization, the events of yesterday afternoon — that the model does not. If the user does not write those things into the prompt, the model cannot use them. The model can produce a fluent, internally coherent, grounded, well-formed answer that is wrong because the question itself was underspecified.

This is the failure mode that experienced LLM users learn to recognize first and explain to others last. From the inside it looks like a model failure: the answer is wrong, so the model must be wrong. From the outside it is a prompt failure: the model gave the most reasonable answer to the question it was actually asked, which differs from the question the user thought they were asking. The gap between the two is the user's environment.

The mechanism is fundamental and not fixable by training. A language model has access to its weights and its context window. It does not have access to the user's calendar, the user's team, the user's organization's coding conventions, the user's medical history, the user's project's constraints, or any of the several hundred other things that shape what counts as a good answer in a real situation. Some of these can be imported into the context — through retrieval, through tool use, through careful prompting, through long-running agent contexts that accumulate state. None of them can be imported automatically.

The diagnostic signature is distinctive. The user reads the answer and feels frustrated; the answer is, on its own terms, fine; what is wrong is that it does not address the real situation. The fix is rarely to ask the model again. The fix is to write down, explicitly, the parts of the environment the model needed to see. Practitioners who become good at LLMs are practitioners who have learned which parts of their environment need importing for which kinds of question.

Walking the ladder

The five axes form a ladder because they are arranged in roughly increasing distance from what the LLM can reliably do on its own. The recommended walk, when an output arrives that strikes you as wrong, is bottom-up:

1. *Structure*: does the output parse, compile, validate against its schema?
2. *Meaning*: read it as a self-contained artifact — does it contradict itself?
3. *Context*: is it appropriate for the situation, audience, and purpose?
4. *Grounding*: are the factual claims true?
5. *Environment*: was the question itself fully specified?

The discipline is that lower-axis failures are typically easier to diagnose and cheaper to remedy than higher-axis ones. A practitioner who jumps straight to "the model hallucinated"

may have missed a context failure that does not require any change to the model at all. Conversely, a practitioner who never considers grounding will deploy systems that are confidently wrong about the world.

Return to the developer with the customer-records function. We walk the ladder.

Structure passes: the function parses and imports. *Meaning* passes: it does not contradict itself; the type signature matches the return. *Context* fails: the codebase uses a particular ORM and the model wrote raw list comprehensions instead. We add the codebase conventions to the prompt and the model writes ORM code.

The function still produces wrong output. We walk again. Structure, meaning, context now all pass. *Grounding* fails: the model called `customers.filter_by_region(r)`, a method that looked plausible but does not exist on the actual ORM. We give the model the ORM definitions, by retrieval, and it picks a real method.

The function still produces wrong output. The methods are real. The output is sensibly wrong: the regions are off. *Environment* fails: in this codebase, "region" is a sales territory id, not a country code. The user assumed the model could see what they meant by region. It could not. The user specifies what region means, and the function works.

The walk shows the discipline. We did not stop at the first axis that passed. We did not jump to the most exotic explanation. We located the failure at the level where the remedy lives, and each fix targeted the right intervention point: prompt content for context, retrieval for grounding, prompt specification for environment.

What the model does not capture

The taxonomy is useful because it is simple. It is also incomplete. Three classes of failure escape it.

The first is *normative*. An output that is structurally fine, internally coherent, situationally appropriate, factually grounded, and environmentally informed but is, by some standard the user holds, simply wrong — offensive, biased, in conflict with values the user expects the system to hold. The five axes ask whether the answer is correct. The normative dimension asks whether the answer is right, and right is not the same thing as correct.

The second is *adversarial*. An output deliberately induced by a malicious prompt, a poisoned retrieval source, or a manipulated tool. These failures cross trust boundaries and need defenses in depth, not diagnostic ladders.

The third is *drift*. An output that was correct yesterday and is wrong today because the world has changed. This is a failure of the system's integration with reality, not of any single output. The remedy lives in monitoring, evaluation, and governance.

The five-axis model is meant for the user-facing case in which a single output is in front of you and you want to know what to do. For everything else, the broader engineering literature on safety, adversarial robustness, and operational monitoring remains the authoritative reference. Hold the ladder loosely. The axes are not orthogonal; failures often span multiple. The ordering is a heuristic, not a theorem. The model is at its most useful in the conversation, not in the algorithm.

Why this matters

The pedagogical use of the model is twofold. For users, it is a diagnostic ladder. For engineers, it is a translation layer between the technical vocabulary of the field — cross-entropy, retrieval, alignment, RLHF — and the user-facing vocabulary of "the model got it wrong." A complaint that arrives as "the bot hallucinated" can, after a one-minute walk through the axes, become "the prompt did not say what region meant, and the model assumed wrong." The fix is no longer in the model and the conversation moves where it belongs.

The model also gives engineering teams a structured way to reason about where to spend their effort. Lower axes call for model-side and decoder-side fixes. Higher axes call for system-side fixes — retrieval, tool use, prompt-construction discipline, agent-context accumulation. A team that wants to improve grounding will work on retrieval, not on the language model. A team that wants to improve environment will work on its prompt-construction discipline, not on retrieval. Knowing which axis you are on tells you which part of the playbook to consult.

This is what the ladder is for. It is not a theory of language. It is a procedure for the moment in which an output is wrong and someone needs to say, in plain English, why.

References

[1] S. Harnad, "The Symbol Grounding Problem," *Physica D: Nonlinear Phenomena*, vol. 42, no. 1–3, pp. 335–346, 1990.

[2] E. M. Bender and A. Koller, "Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 5185–5198.

[3] Z. Ji *et al.*, "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

Lodewijk Bonebakker is the author of Large Language Models as Engineered Systems*, a forthcoming primer for engineers on the systems, statistical, and human-factors dimensions of modern LLMs. He can be reached at* `focus2flow@bonebakker.org`.